FUNCTION CALLS

Functions are very important to any language. So I thought it would be good to declare them all before we begin the program.

I made a separate stack called funcs and to mark to the top of this funcs-tos.

```
section .bss

    funcs:      resq 0x64       ; enough for 100 functions
    funcs-tos:  resq 0x01
```

funcs is a stack (really just a basic array) that holds info about a function. I allocated 2x resq (16 bytes) for each time I came across a function. The 1st 8 bytes will store the name of the function and the next 8 bytes will store the position of the 1st token of the actual code.

The best way I can explain this is by showing you this code:

```
func: greeting

    "hello world!" print-str
    return

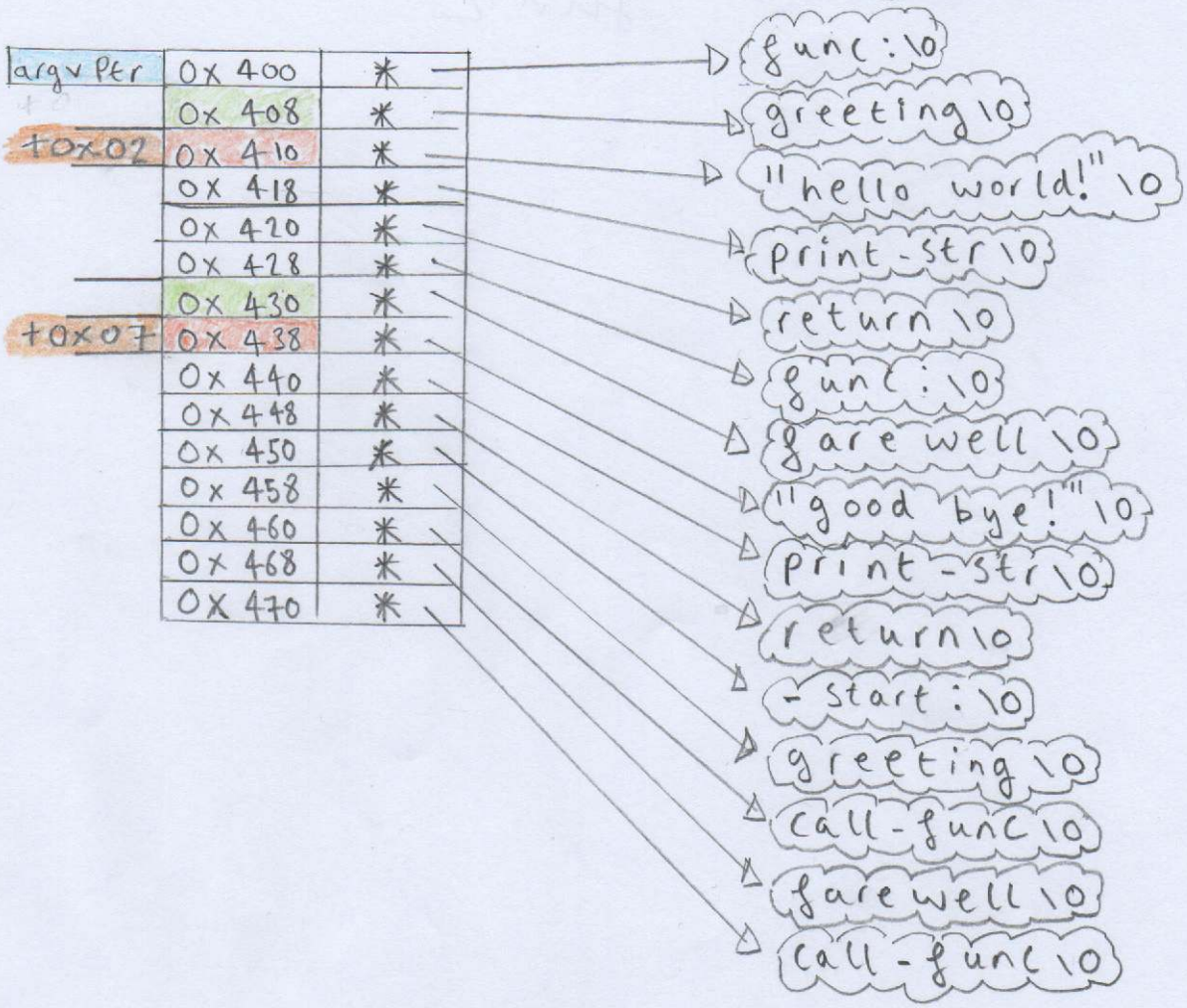
func: farewell

    "goodbye!" print-str
    return

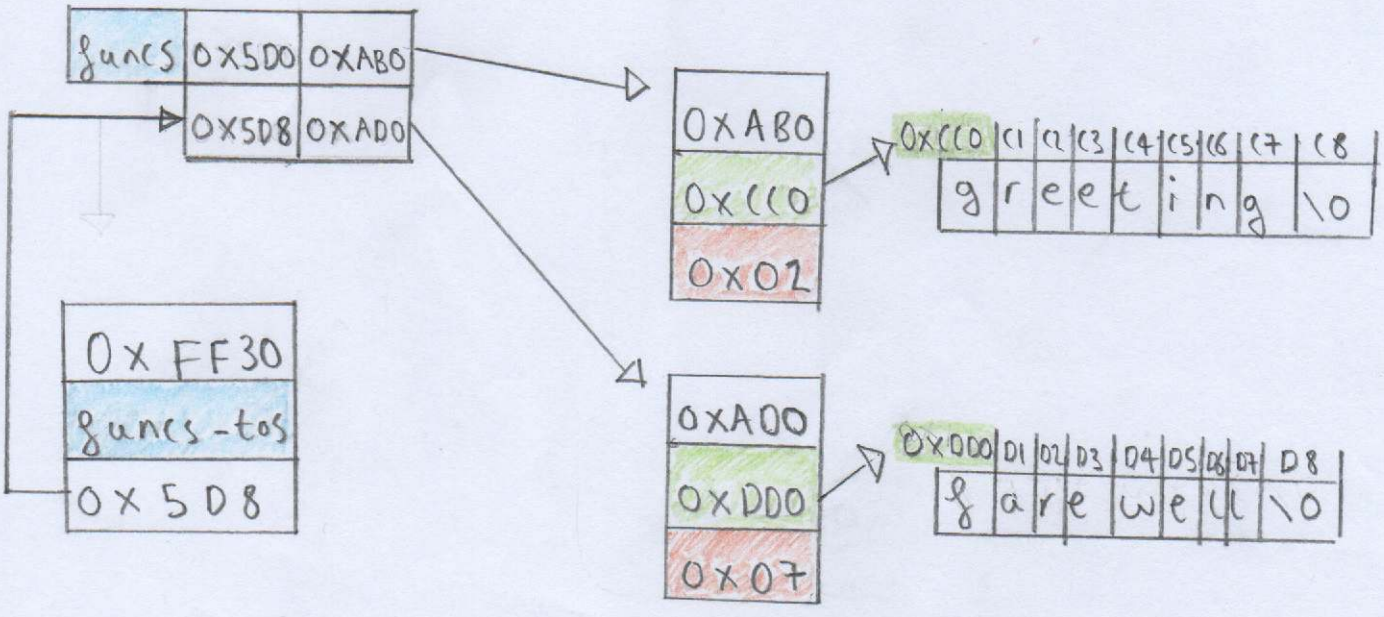
_start:

    greeting  call_func
    farewell  call_func
```

When we parse the code into the argvPtr, it looks like this:



To set up all the functions, I scan the argvPtr and when I match the string 'func:', I know the next resq will be the function name. The resq which is after the name is where the code begins.



To set up all the functions, I scan argvPtr and when I match the string 'func:' I know the next resq will be the function name. The resq which is after the name, is where the code begins.

In NASM wherever a function is called, the rip register makes a note of where to resume after the function has ended. A pointer to the next instruction is pushed onto the stack.

I did the same thing and made a note of the next position in argvPtr where the program will resume from. I made another stack for this and called it res-rip and res-rip-tos in .bss section.

When a function is called res-rip is updated with the resume position in our code. Then, at the end of the function when 'return' is found, the res-rip-tos pops off. It's the value which the program carries on working from this point.

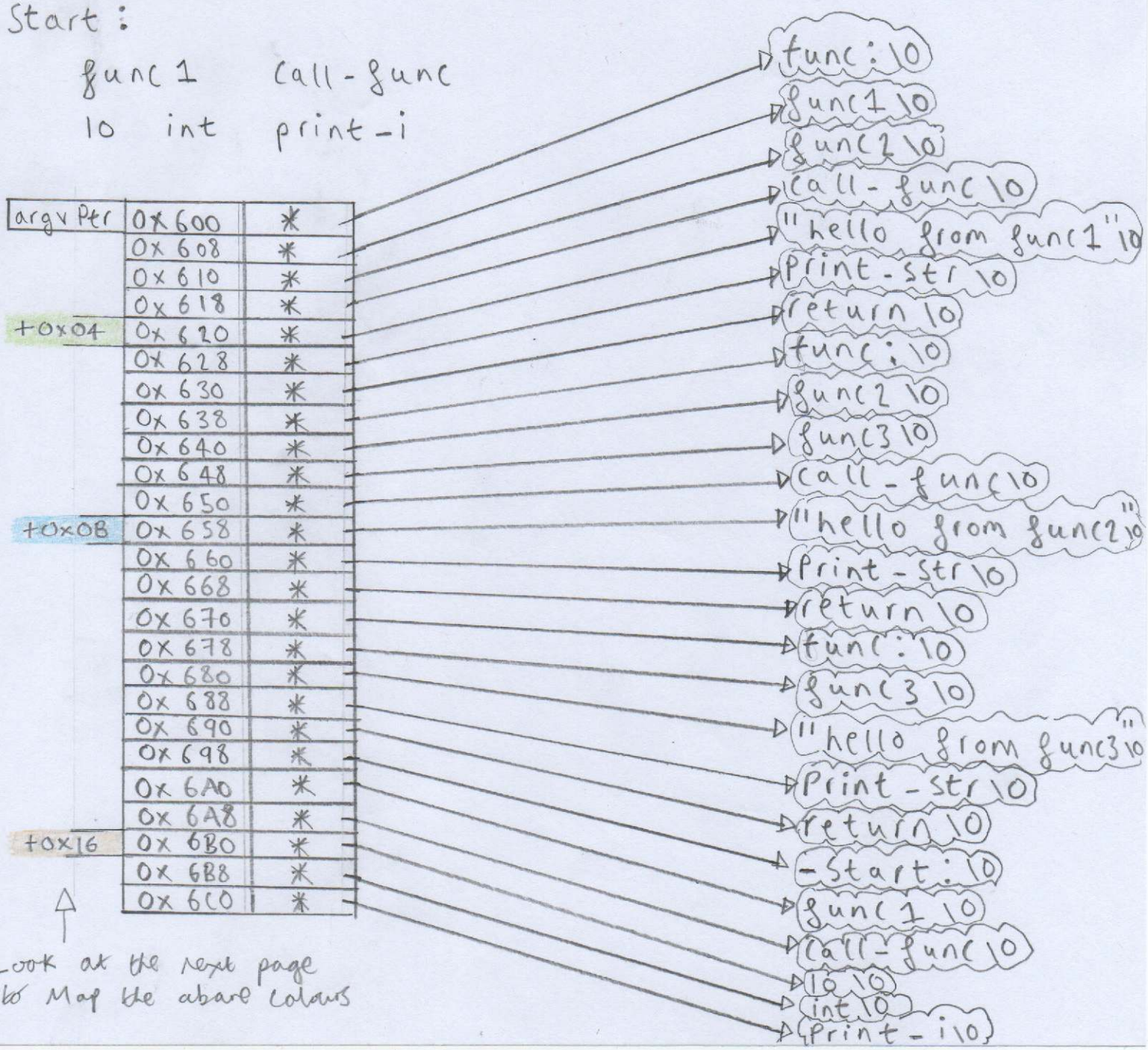
On the next page there is a simple program and I've put the argvPtr with all the tokens below it.

I've also colour coded the resume points once the function call has ended, this happens when the 'return' opcode is found.

```
func:   func1
    func2    call-func
    "hello from func1"  print-str
    return

func:   func2
    func3    call-func
    "hello from func2"  print-str
    return

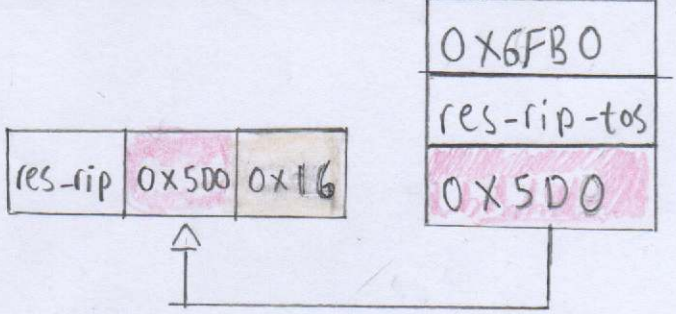
func:   func3
    "hello from func3"  print-str
    return
-start:
    func1    call-func
    10  int  print-i
```



Look at the next page to Map the above colours

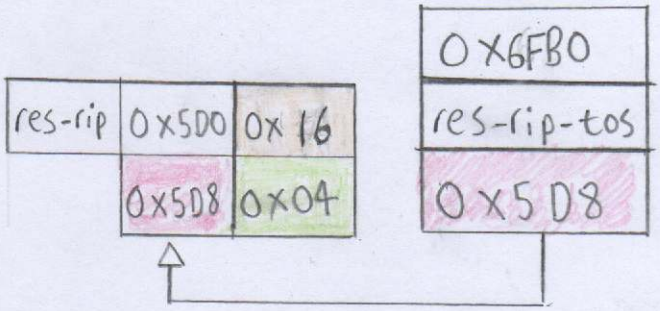
In _start : we come across the call-func operand :

```
func1   call-func
10  int  print-i
```



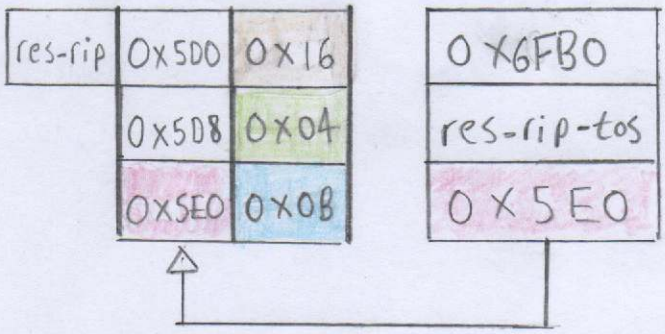
Inside func1 we come across call-func opeand :

```
func2   call-func
"hello from func1"  print-str
```



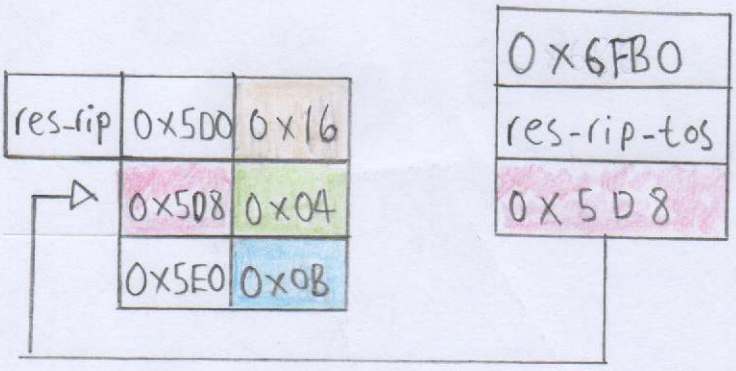
Inside func2 we come across call-func opeand :

```
func3   call-func
"hello from func2"  print-str
```



Inside func3 we come across return operand . Now I have to find where to resume processing , res-rip-tos has the value . It's easy to find the index I need to jump to , in my argV Ptr.

```
Mov   r9, [res-rip-tos]
Mov   r9, [r9]                ; Now r9 has value 0x0B
Sub   qword [res-rip-tos] ; move to next position
                                  , 0x08
```

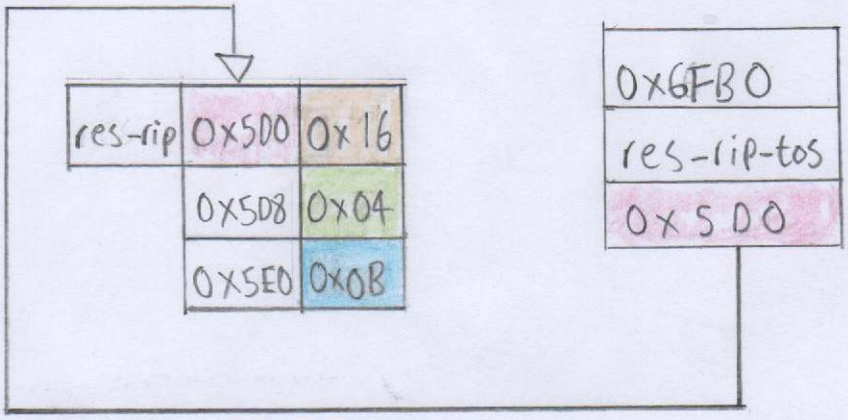


Now I can resume processing at [argvPtr + 19 * 0x08] Position. This will be "hello from func2" token.

Inside func2 we come across return operand. Again, I use the same steps to find out where to resume from.

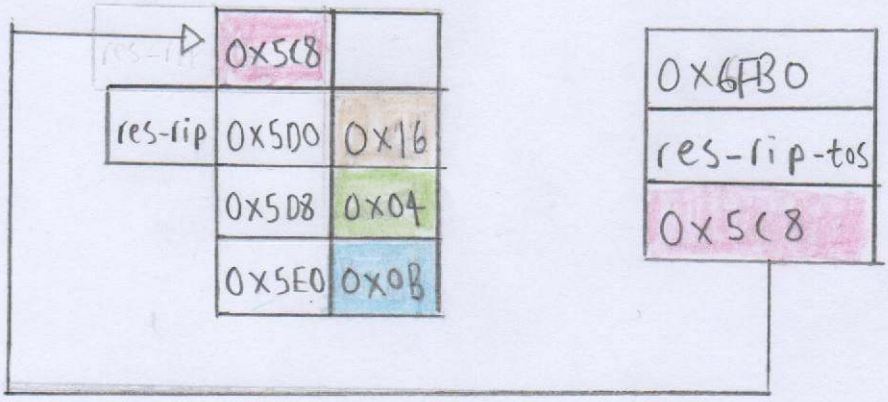
Now I resume processing at the token:
"hello from func1" token

```
Sub   qword [res-rip-tos], 0x08 ; Move to next position
```



Inside func1 we come across return operand. I repeat the same steps again. This time we resume from _start: at token 10.

```
Sub   qword [res-rip-tos], 0x08 ; Move to next position
```



res-rip-tos now points to 0x08 bytes before res-rip, this means we must be in the _start: function.